



Linux Fundamentals

0%



Section 24 / 30

Network Configuration

As a penetration tester, one of the essential skills is configuring and managing network settings on Linux systems. Mastering this allows us to efficiently set up testing environments, manipulate network traffic, and identify or exploit vulnerabilities. A solid understanding of Linux network configuration gives us the ability to tailor our testing approach to suit specific needs, helping optimize both our testing procedures and results.

One of the primary tasks in network configuration is managing network interfaces. This involves assigning IP addresses, configuring network devices such as routers and switches, and setting up various network protocols. A deep understanding of network protocols, including TCP/IP (the core protocol suite for Internet communications), DNS (domain name resolution), DHCP (for dynamic IP address allocation), and FTP (file transfer), is critical. We must also be familiar with different types of network interfaces—whether wired or wireless—and be able to troubleshoot connectivity issues.

Network Access Control

Another vital component of network configuration is network access control (**NAC**). As penetration testers, we need to be well-versed in how NAC can enhance network security and the

various technologies available. Key NAC models include:

Type	Description
Discretionary Access Control (DAC)	This model allows the owner of the resource to set permissions for who can access it.
Mandatory Access Control (MAC)	Permissions are enforced by the operating system, not the owner of the resource, making it more secure but less flexible.
Role-Based Access Control (RBAC)	Permissions are assigned based on roles within an organization, making it easier to manage user privileges.

Configuring Linux network devices for NAC involves setting up security policies like **SELinux** (Security-Enhanced Linux), **AppArmor** profiles for application security, and using **TCP wrappers** to control access to services based on IP addresses. More about this in the future sections.

Tools such as **syslog** , **rsyslog** , **ss** (for socket statistics), **lsof** (to list open files), and the **ELK stack** (Elasticsearch, Logstash, and Kibana) can be used to monitor and analyze network traffic. These tools help identify anomalies, potential information disclosure/expose, security breaches, and other critical network issues.

Think of network configuration in Linux like building and securing a large office building. Configuring network interfaces is like setting up the wiring and infrastructure, ensuring that each room (network device) has a working connection. NAC is like managing the building's security where some rooms are open to everyone (**DAC**), while others are only accessible to certain

people based on strict rules (`MAC` or `RBAC`). Monitoring network traffic is similar to installing surveillance cameras and alarms, keeping an eye on who is moving through the building, and troubleshooting is like having a toolkit on hand to fix any issues—whether a broken connection (`ping`), a faulty lock (`nslookup`), or a vulnerable entrance (`nmap`). We will explore NAC and the tools in greater detail a bit later in this section.

Configuring Network Interfaces

When working with Ubuntu, you can configure local network interfaces using the `ifconfig` or the `ip` command. These powerful commands allow us to view and configure our system's network interfaces. Whether we're looking to make changes to our existing network setup or need to check on the status of our interfaces, these commands can greatly simplify the process. Moreover, developing a firm grasp on the intricacies of network interfaces is an essential ability in the modern, interconnected world. With the rapid advancement of technology and the increasing reliance on digital communication, having a comprehensive knowledge of how to work with network interfaces can enable you to navigate the diverse array of networks that exist nowadays effectively.

One way to obtain information regarding network interfaces, such as IP addresses, netmasks, and status, is by using the `ifconfig` command. By executing this command, we can view the available network interfaces and their respective attributes in a clear and organized manner. This information can be particularly useful when troubleshooting network connectivity issues or setting up a new network configuration. It should be noted that the `ifconfig` command has been deprecated in newer versions of Linux and replaced by the `ip` command, which offers

more advanced features. Nevertheless, the `ifconfig` command is still widely used in many Linux distributions and continues to be a reliable tool for network management.

Network Settings

shellsession

```
cry011t3@htb:~$ ifconfig
```

```
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 178.62.32.126 netmask 255.255.192.0 broadcast 178.62.63.255
    inet6 fe80::88d9:faff:fecf:797a prefixlen 64 scopeid 0x20<link>
    ether 8a:d9:fa:cf:79:7a txqueuelen 1000 (Ethernet)
    RX packets 7910 bytes 717102 (700.2 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 7072 bytes 24215666 (23.0 MiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
eth1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.106.0.66 netmask 255.255.240.0 broadcast 10.106.15.255
    inet6 fe80::b8ab:52ff:fe32:1f33 prefixlen 64 scopeid 0x20<link>
    ether ba:ab:52:32:1f:33 txqueuelen 1000 (Ethernet)
    RX packets 14 bytes 1574 (1.5 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 15 bytes 1700 (1.6 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

```
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
```

```
RX packets 15948  bytes 24561302 (23.4 MiB)
RX errors 0  dropped 0  overruns 0  frame 0
TX packets 15948  bytes 24561302 (23.4 MiB)
TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
```

```
cry011t3@htb:~$ ip addr
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether 8a:d9:fa:cf:79:7a brd ff:ff:ff:ff:ff:ff
    altname enp0s3
    altname ens3
    inet 178.62.32.126/18 brd 178.62.63.255 scope global dynamic eth0
        valid_lft 85274sec preferred_lft 85274sec
    inet6 fe80::88d9:faff:febf:797a/64 scope link
        valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group
default qlen 1000
    link/ether ba:ab:52:32:1f:33 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    altname ens4
    inet 10.106.0.66/20 brd 10.106.15.255 scope global dynamic eth1
```

```
valid_lft 85274sec preferred_lft 85274sec
inet6 fe80::b8ab:52ff:fe32:1f33/64 scope link
valid_lft forever preferred_lft forever
```

When it comes to activating network interfaces, `ifconfig` and `ip` commands are two commonly used tools. These commands allow users to modify and activate settings for a specific interface, such as `eth0`. We can adjust the network settings to suit our needs by using the appropriate syntax and specifying the interface name.

Activate Network Interface

```
athane@htb[/htb]$ sudo ifconfig eth0 up      # OR
athane@htb[/htb]$ sudo ip link set eth0 up
```

shellsession

One way to allocate an IP address to a network interface is by utilizing the `ifconfig` command. We must specify the interface's name and IP address as arguments to do this. This is a crucial step in setting up a network connection. The IP address serves as a unique identifier for the interface and enables the communication between devices on the network.

Assign IP Address to an Interface

```
athane@htb[/htb]$ sudo ifconfig eth0 192.168.1.2
```

shellsession

To set the netmask for a network interface, we can run the following command with the name of the interface and the netmask:

Assign a Netmask to an Interface

```
athane@htb[/htb]$ sudo ifconfig eth0 netmask 255.255.255.0
```

shellsession

When we want to set the default gateway for a network interface, we can use the `route` command with the `add` option. This allows us to specify the gateway's IP address and the network interface to which it should be applied. By setting the default gateway, we are designating the IP address of the router that will be used to send traffic to destinations outside the local network. Ensuring that the default gateway is set correctly is important, as incorrect configuration can lead to connectivity issues.

Assign the Route to an Interface

```
athane@htb[/htb]$ sudo route add default gw 192.168.1.1 eth0
```

shellsession

When configuring a network interface in Linux, it is often necessary to set Domain Name System (`DNS`) servers to ensure proper network functionality. DNS servers are responsible for translating domain names (like example.com) into IP addresses, which allows devices to locate and connect to one another on the internet. Proper DNS configuration is crucial for enabling devices to access websites, online services, and other networked resources. Without correctly configured DNS

servers, devices may experience issues such as the inability to resolve domain names, leading to network connectivity problems.

On Linux systems, this can be achieved by updating the `/etc/resolv.conf` file, which is a simple text file containing the system's DNS information. By adding the appropriate DNS server addresses (Google's public DNS - `8.8.8.8` or `8.8.4.4`), the system can correctly resolve domain names to IP addresses, ensuring smooth communication over the network.

Editing DNS Settings

```
athane@htb[/htb]$ sudo vim /etc/resolv.conf
```

shellsession

`/etc/resolv.conf`

```
nameserver 8.8.8.8  
nameserver 8.8.4.4
```

txt

After completing the necessary modifications to the network configuration, it is essential to ensure that these changes are saved to persist across reboots. This can be achieved by editing the `/etc/network/interfaces` file, which defines network interfaces for Linux-based operating systems. Thus, it is vital to save any changes made to this file to avoid any potential issues with network connectivity.

It's important to note that changes made directly to the `/etc/resolv.conf` file are not persistent across reboots or network configuration changes. This is because the file may be automatically overwritten by network management services like `NetworkManager` or `systemd-resolved`. To make DNS changes permanent, you should configure DNS settings through the appropriate network management tool, such as editing network configuration files or using network management utilities that store persistent settings.

Editing Interfaces

```
athane@htb[/htb]$ sudo vim /etc/network/interfaces
```

shellsession

This will open the `interfaces` file in the vim editor. We can add the network configuration settings to the file like this:

`/etc/network/interfaces`

```
auto eth0
iface eth0 inet static
    address 192.168.1.2
    netmask 255.255.255.0
    gateway 192.168.1.1
    dns-nameservers 8.8.8.8 8.8.4.4
```

txt

By setting the `eth0` network interface to use a static IP address of `192.168.1.2`, with a netmask of `255.255.255.0` and a default gateway of `192.168.1.1`, we can ensure that your

network connection remains stable and reliable. Additionally, by specifying DNS servers of `8.8.8.8` and `8.8.4.4`, we can ensure that our computer can easily access the internet and resolve domain names. Once we have made these changes to the configuration file, saving the file and exiting the editor is important. After that, we must restart the networking service to apply the changes.

Restart Networking Service

```
athane@htb[/htb]$ sudo systemctl restart networking
```

shellsession

Network Access Control

Network access control (NAC) is a crucial component of network security, especially in today's era of increasing cyber threats. As a penetration tester, it is vital to understand the significance of NAC in protecting the network and the various NAC technologies that can be utilized to enhance security measures. NAC is a security system that ensures that only authorized and compliant devices are granted access to the network, preventing unauthorized access, data breaches, and other security threats. By implementing NAC, organizations can be confident in their ability to protect their assets and data from cybercriminals who always seek to exploit system vulnerabilities. The following are the different NAC technologies that can be used to enhance security measures:

- Discretionary access control (DAC)
- Mandatory access control (MAC)
- Role-based access control (RBAC)

These technologies are designed to provide different levels of access control and security. Each technology has its unique characteristics and is suitable for different use cases. As a penetration tester, it is essential to understand these technologies and their specific use cases to test and evaluate the network's security effectively.

Discretionary Access Control

DAC is a crucial component of modern security systems as it helps organizations provide access to their resources while managing the associated risks of unauthorized access. It is a widely used access control system that enables users to manage access to their resources by granting resource owners the responsibility of controlling access permissions to their resources. This means that users and groups who own a specific resource can decide who has access to their resources and what actions they are authorized to perform. These permissions can be set for reading, writing, executing, or deleting the resource.

Mandatory Access Control

MAC is used in infrastructure that provides more fine-grained control over resource access than DAC systems. Those systems define rules that determine resource access based on the resource's security level and the user's security level or process requesting access. Each resource is assigned a security label that identifies its security level, and each user or process is assigned a security clearance that identifies its security level. Access to a resource is only granted if the user's or process's security level is equal to or greater than the security level of the resource. MAC is often used in operating systems and applications that require a high level of security, such as military or government systems, financial systems, and healthcare systems. MAC systems are designed to prevent unauthorized access to resources and minimize the impact of security breaches.

Role-based Access Control

RBAC assigns permissions to users based on their roles within an organization. Users are assigned roles based on their job responsibilities or other criteria, and each role is granted a set of permissions that determine the actions they can perform. RBAC simplifies the management of access permissions, reduces the risk of errors, and ensures that users can access only the resources necessary to perform their job functions. It can restrict access to sensitive resources and data, limit the impact of security breaches, and ensure compliance with regulatory requirements. Compared to Discretionary Access Control (DAC) systems, RBAC provides a more flexible and scalable approach to managing resource access. In an RBAC system, each user is assigned one or more roles, and each role is assigned a set of permissions that define the user's actions. Resource access is granted based on the user's assigned role rather than their identity or ownership of the resource. RBAC systems are typically used in environments with many users and resources, such as large organizations, government agencies, and financial institutions.

Monitoring

Network monitoring involves capturing, analyzing, and interpreting network traffic to identify security threats, performance issues, and suspicious behavior. The primary goal of analyzing and monitoring network traffic is identifying security threats and vulnerabilities. For example, as penetration testers, we can capture credentials when someone uses an unencrypted connection and tries to log in to an FTP server. As a result, we will obtain this user's credentials that might help us to infiltrate the network even further or escalate our privileges to a higher level. In short, by analyzing network traffic, we can gain insights into network behavior and identify patterns that may indicate security threats. Such analysis includes detecting suspicious network activity, identifying malicious traffic, and identifying potential security risks. However, we cover this vast

topic in the **Intro to Network Traffic Analysis** module, where we use several tools for network monitoring on Linux systems like Ubuntu and Windows systems, like Wireshark, tshark, and Tcpdump.

Troubleshooting

Network troubleshooting is an essential process that involves diagnosing and resolving network issues that can adversely affect the performance and reliability of the network. This process is critical for ensuring the network operates optimally and avoiding disruptions that could impact business operations during our penetration tests. It also involves identifying, analyzing, and implementing solutions to resolve problems. Such problems include connectivity problems, slow network speeds, and network errors. Various tools can help us identify and resolve issues regarding network troubleshooting on Linux systems. Some of the most commonly used tools include:

1. Ping
2. Traceroute
3. Netstat
4. Tcpdump
5. Wireshark
6. Nmap

By using these tools and others like them, we can better understand how the network functions and quickly diagnose any issues that may arise. For example, `ping` is a command-line tool used to test connectivity between two devices. It sends packets to a remote host and measures the time to return them. To use `ping`, we can enter the following command:

Ping

shellsession

```
athane@htb[/htb]$ ping <remote_host>
```

For example, pinging the Google DNS server will send ICMP packets to the Google DNS server and display the response times.

shellsession

```
athane@htb[/htb]$ ping 8.8.8.8
```

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.  
64 bytes from 8.8.8.8: icmp_seq=1 ttl=119 time=1.61 ms  
64 bytes from 8.8.8.8: icmp_seq=2 ttl=119 time=1.06 ms  
64 bytes from 8.8.8.8: icmp_seq=3 ttl=119 time=0.636 ms  
64 bytes from 8.8.8.8: icmp_seq=4 ttl=119 time=0.685 ms  
^C  
--- 8.8.8.8 ping statistics ---  
4 packets transmitted, 4 received, 0% packet loss, time 3017ms  
rtt min/avg/max/mdev = 0.636/0.996/1.607/0.388 ms
```

Another tool is the `tracert`, which traces the route packets take to reach a remote host. It sends packets with increasing Time-to-Live (TTL) values to a remote host and displays the IP addresses of the devices that the packets pass through. For example, to trace the route to www.inlanefreight.com, we would enter the following command:

Traceroute

```
athane@htb[/htb]$ traceroute www.inlanefreight.com
```

```
traceroute to www.inlanefreight.com (134.209.24.248), 30 hops max, 60 byte packets
 1  * * *
 2  10.80.71.5 (10.80.71.5)  2.716 ms  2.700 ms  2.730 ms
 3  * * *
 4  10.80.68.175 (10.80.68.175)  7.147 ms  7.132 ms  10.80.68.161 (10.80.68.161)
    7.393 ms
```

The output provides a detailed view of the path packets take to reach the Inlanefreight web server, located at www.inlanefreight.com with the IP address 134.209.24.248. It displays the IP addresses of all intermediary devices the packets traverse. Each line of the traceroute output contains valuable information, offering insights into the network route and performance.

When setting up a network connection, it's important to specify the destination host and IP address. In this example, the destination host is 134.209.24.248, and the maximum number of hops allowed is 30. This ensures that the connection is established efficiently and reliably. By providing this information, the system can route traffic to the correct destination and limit the number of intermediate stops the data needs to make.

The second line shows the first hop in the traceroute, which is the local network gateway with the IP address 10.80.71.5, followed by the next three columns show the time it took for each of the three packets sent to reach the gateway in milliseconds (2.716 ms, 2.700 ms, and 2.730 ms).

Next, we see the second hop in the traceroute. However, there was no response from the device at that hop, indicated by the three asterisks instead of the IP address. This could mean the device

is down, blocking ICMP traffic, or a network issue caused the packets to drop.

In the fourth line, we can see the third hop in the traceroute, consisting of two devices with IP addresses 10.80.68.175 and 10.80.68.161, and again the next three columns show the time it took for each of the three packets to reach the first device (7.147 ms, 7.132 ms, and 7.393 ms).

Netstat

Netstat is used to display active network connections and their associated ports. It can be used to identify network traffic and troubleshoot connectivity issues. To use **netstat**, we can enter the following command:

```
athane@htb[/htb]$ netstat -a
```

Active Internet connections (servers and established)

Proto	Recv-Q	Send-Q	Local Address	Foreign Address	State
tcp	0	0	localhost:5901	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:sunrpc	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:http	0.0.0.0:*	LISTEN
tcp	0	0	0.0.0.0:ssh	0.0.0.0:*	LISTEN
...SNIP...					

We can expect to receive detailed information about each connection when using this tool. This includes the protocol used, the number of bytes received and sent, IP addresses, port numbers of both local and remote devices, and the current connection state. The output provides valuable insights into the network activity on the system, highlighting four specific connections currently active and listening on specific ports. These connections include the VNC remote desktop

software, the Sun Remote Procedure Call service, the HTTP protocol for web traffic, and the SSH protocol for secure remote shell access. By knowing which ports are used by which services, users can quickly identify any network issues and troubleshoot accordingly. The most common network issues we will encounter during our penetration tests are as follows:

- Network connectivity issues
- DNS resolution issues (it's always about DNS)
- Loss of data packets
- Network performance issues

The most common causes for them are:

- Incorrectly configured firewalls or routers,
- damaged network cables or connections,
- incorrect network settings,
- hardware failures,
- incorrect DNS server settings or DNS server failures
- incorrectly configured DNS entries,
- network congestion,
- outdated network hardware or incorrectly configured network settings,
- unpatched software or firmware and missing security controls.

Understanding these common network issues and their causes is important for effectively identifying and exploiting vulnerabilities in network systems during our testing.

Hardening

Several mechanisms are highly effective in securing Linux systems in keeping our and other companies' data safe. Three such mechanisms are SELinux, AppArmor, and TCP wrappers. These tools are designed to safeguard Linux systems against various security threats, from unauthorized access to malicious attacks. This is critical not only during penetration tests, where systems are intentionally stressed to uncover vulnerabilities, but also in real-world scenarios where an actual compromise could have serious consequences (few situations are as severe as a real-life breach.) By implementing these security measures and ensuring that we set up corresponding protection against potential attackers, we can significantly reduce the risk of data leaks and ensure our systems remain secure. While these tools share some similarities, they also have important differences.

Security-Enhanced Linux

Security-Enhanced Linux (SELinux) is a mandatory access control (MAC) system integrated into the Linux kernel. It provides fine-grained control over access to system resources and applications by enforcing security policies. These policies define the permissions for each process and file on the system, significantly limiting the damage that a compromised process or service can do. SELinux operates at a low level, and though it offers strong security, it can be complex to configure and manage due to its granular controls.

AppArmor

Like SELinux, AppArmor is a MAC system that controls access to system resources and applications, but it operates in a simpler, more user-friendly manner. AppArmor is implemented as a Linux Security Module (LSM) and uses application profiles to define what resources an application can access. While it may not provide the same level of fine-grained control as SELinux,

AppArmor is often easier to configure and is generally considered more straightforward for day-to-day use.

TCP Wrappers

TCP wrappers are a host-based network access control tool that restricts access to network services based on the IP address of incoming connections. When a network request is made, TCP wrappers intercept it, checking the request against a list of allowed or denied IP addresses. This is a simple yet effective way to control access to services, especially for blocking unauthorized systems from accessing networked resources. While it does not offer the fine-grained control of SELinux or AppArmor, TCP wrappers are an excellent tool for basic network-level protection.

Regarding similarities, the three security mechanisms share the common goal of ensuring the safety and security of Linux systems. In addition to providing extra protection, they can restrict access to resources and services, thus reducing the risk of unauthorized access and data breaches. It's also worth noting that these mechanisms are readily available as part of most Linux distributions, making them accessible to us to enhance their systems' security. Furthermore, these mechanisms can be easily customized and configured using standard tools and utilities, making them a convenient choice for Linux users.

Although both **SELinux** and **AppArmor** are MAC systems that provide fine-grained control, they work in different ways. SELinux is deeply integrated into the kernel and offers more detailed security controls, but it can be more complex to configure and maintain. In contrast, AppArmor operates as a kernel module and uses profile-based security, making it easier to manage, though it may not offer the same level of granularity as SELinux.

On the other hand, **TCP wrappers** focus on controlling access to network services based on client IP addresses, which makes it simpler but limited to network-level access control. It doesn't offer the broader system resource protections that SELinux and AppArmor provide, but it's useful for restricting access to services from unauthorized systems.

Setting Up

As we navigate the world of Linux, we inevitably encounter a wide range of technologies, applications, and services that we need to become familiar with. This is a crucial skill, particularly if we work in cybersecurity and strive to improve our expertise continuously. For this reason, we highly recommend dedicating time to learning about configuring important security measures such as **SELinux**, **AppArmor**, and **TCP wrappers** on your own. By taking on this (optional but highly efficient) challenge, you'll deepen your understanding of these technologies, build up your problem-solving skills, and gain valuable experience that will serve you well in the future. We highly recommend to use a personal VM and make snapshots before making changes.

When it comes to implementing cybersecurity measures, there is no one-size-fits-all approach. It is important to consider the specific information you want to protect and the tools you will use to do so. However, you can practice and implement several optional tasks with others in the Discord channel to increase your knowledge and skills in this area. By taking advantage of the helpfulness of others and sharing your own expertise, you can deepen your understanding of cybersecurity and help others do the same. Remember, explaining concepts to others is essential to teaching and learning.

SELinux

1. Install SELinux on your VM.
2. Configure SELinux to prevent a user from accessing a specific file.
3. Configure SELinux to allow a single user to access a specific network service but deny access to all others.
4. Configure SELinux to deny access to a specific user or group for a specific network service.

AppArmor

5. Configure AppArmor to prevent a user from accessing a specific file.
6. Configure AppArmor to allow a single user to access a specific network service but deny access to all others.
7. Configure AppArmor to deny access to a specific user or group for a specific network service.

TCP Wrappers

8. Configure TCP wrappers to allow access to a specific network service from a specific IP address.

9. Configure TCP wrappers to deny access to a specific network service from a specific IP address.
10. Configure TCP wrappers to allow access to a specific network service from a range of IP addresses.

Connect to HTB



24 / 30 Sections



Mark Complete & Next